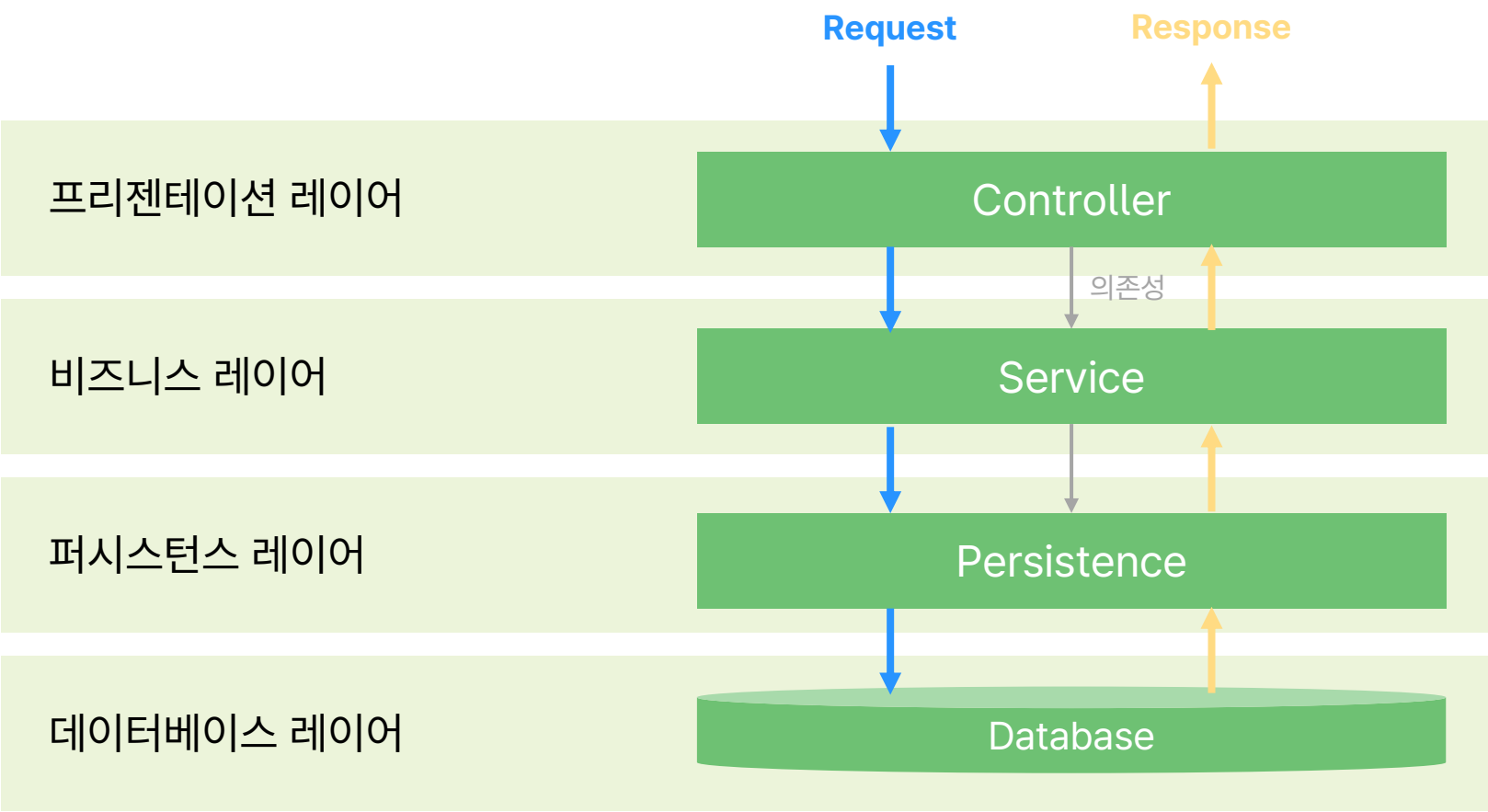
웹 개발자 부트캠프 과정

CodingOn

Layered Architecture

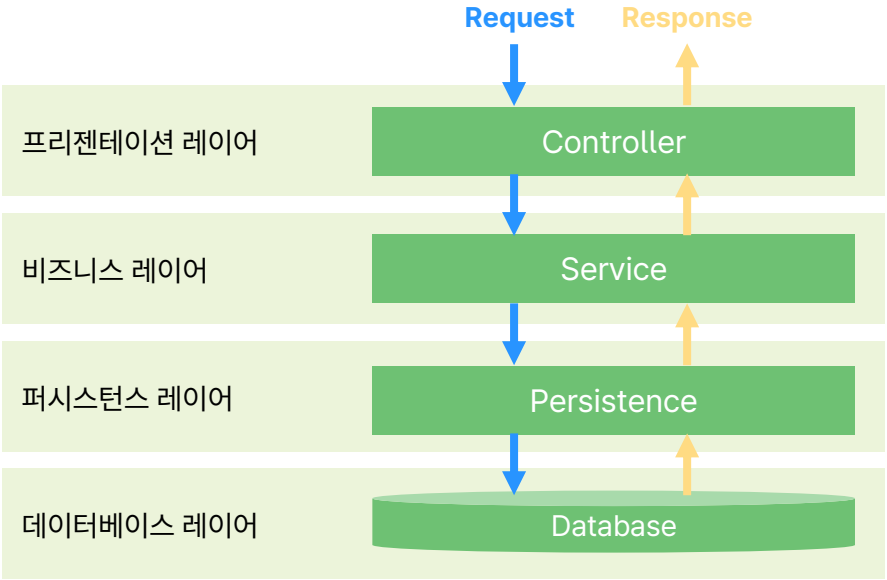


Layered Architecture

- 애플리케이션을 몇 가지 계층으로 나누어 만드는 아키텍처

- 각 계층은 특정 기능에 집중
- 추상화: 상위 계층은 하위 계층의 구체적인 구현을 알 필요가 없음
- 느슨한 결합: 계층 간 의존성을 최소화 하여 유지보수와 확장성 향상

- 단방향 의존성
  - 프리젠테이션 계층은 비즈니스 계층에만 의존성을 가짐
  - 비즈니스 계층은 퍼시스턴스 계층에만 의존성을 가짐
  - 계층이 분리되어 있어 복잡도가 감소하고, 테스트가 쉬움

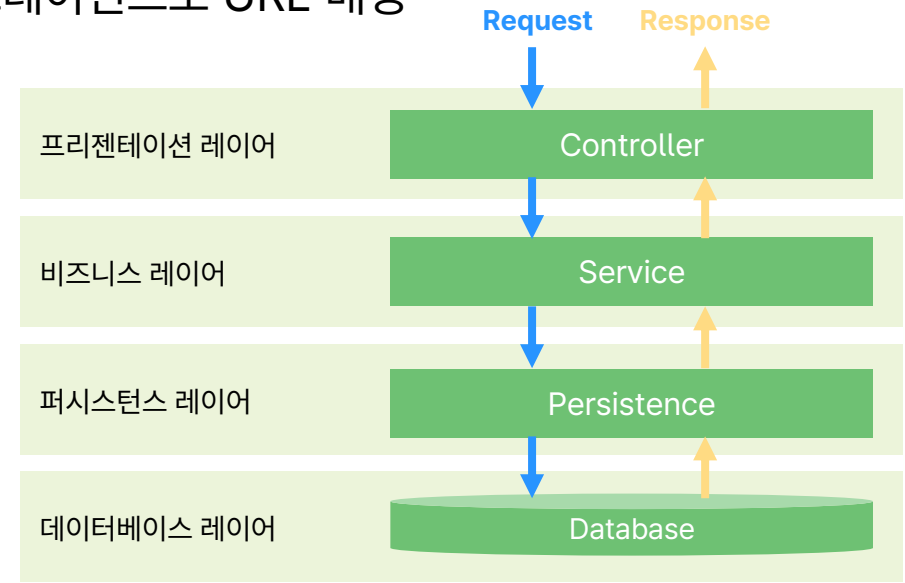


Presentation Layer

- 스프링부트의 controller 패키지에 해당하며, HTTP 요청을 처리하고 응답을 반환

- 주요 구성 요소
  - @Controller 또는 @RestController 어노테이션이 붙은 클래스
  - @RequestMapping, @GetMapping, @PostMapping 등의 어노테이션으로 URL 매핑

- 클라이언트 요청 수신 및 파라미터 추출
- 입력 데이터 유효성 검사
- Service 계층 호출 및 비즈니스 로직 실행 요청
- 데이터를 클라이언트에 적합한 형태(JSON, XML 등)로 반환

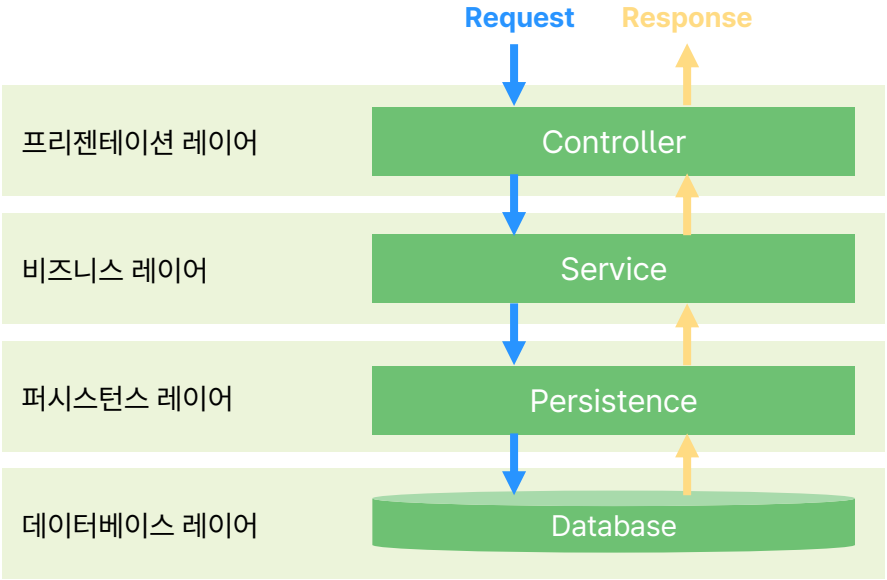


Business Layer

- 스프링부트의 service 패키지에 해당하며, 비즈니스 로직을 구현하거나 트랜잭션 관리

- 주요 구성 요소
  - @Service 어노테이션이 붙은 클래스

- 복잡한 비즈니스 로직 구현
- 여러 Repository 의 데이터를 조합하여 처리
- 필요시 DTO 와 Domain 객체간 변환

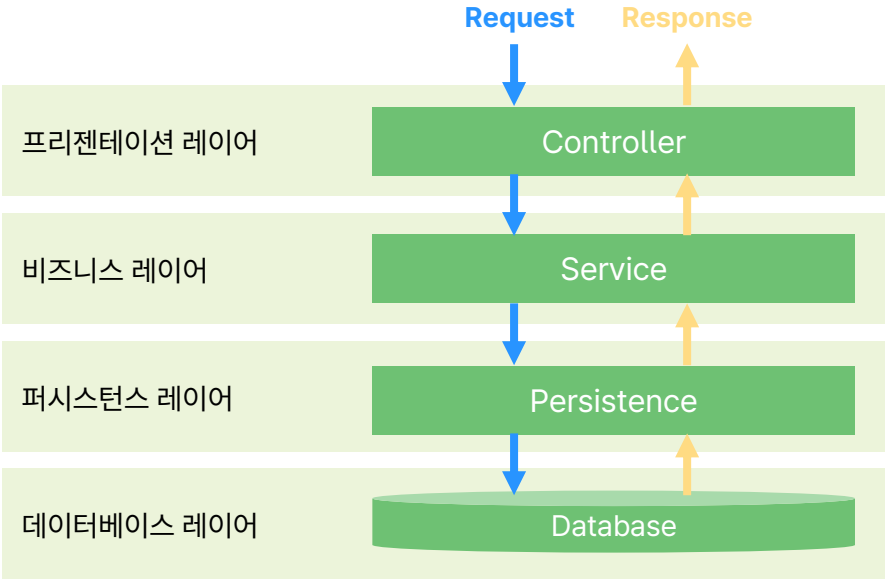


Persistence Layer

- 스프링부트의 repository 패키지에 해당하며, 데이터베이스와의 상호작용 및 데이터 접근 로직 구현

- 주요 구성 요소
  - @Repository 어노테이션이 붙은 인터페이스 또는 클래스
  - Spring Data JPA 사용 시 JpaRepository를 상속

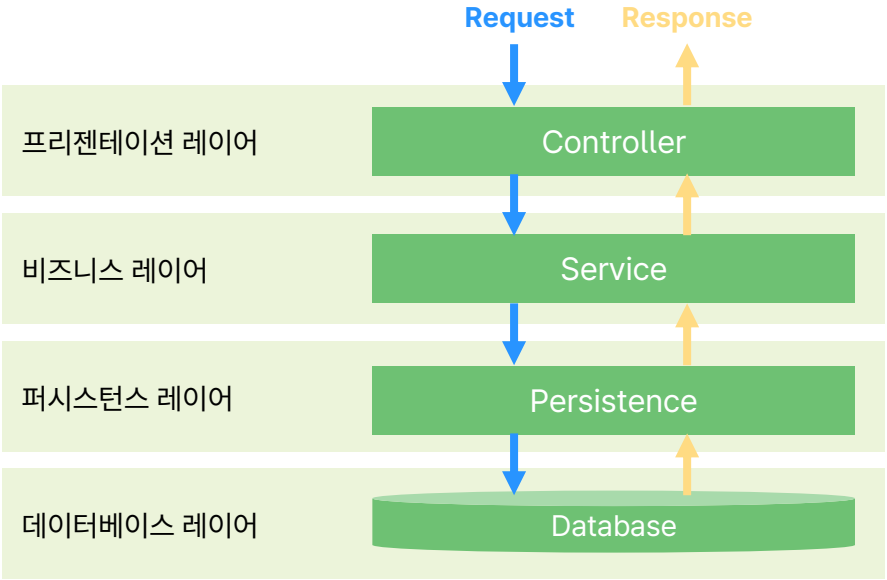
- CRUD 연산 수행
- 쿼리 메소드 또는 @Query 어노테이션을 이용한 복잡한 쿼리 실행
- 데이터베이스 특정 예외를 Spring 의 일반 예외로 변환



Database Layer

- 스프링 프레임워크에서 직접적으로 관리하지 않는 영역이지만, 애플리케이션과 밀접하게 관련되어 있음

- 주요 구성 요소
  - 실제 데이터베이스 시스템 (MySQL, PostgreSQL 등)
  - JPA Entity 클래스 (데이터베이스 테이블과 매핑)



JPA

JPA 란?

- Java Persistence API
- 자바 진영의 ORM 기술 표준
- JPA 가 개발자 대신 적합한 SQL을 생성하고 DB 에 전달하고, 객체를 자동으로 Mapping 해주기에 SQL 을 직접 작성할 필요가 없다.
- ex. Hibernate (JPA 를 구현한 대표적 오픈소스)

JPA 장단점

- 장점
  - 생산성이 뛰어나고 유지보수가 용이하다.
  - DBMS에 대한 종속성이 줄어든다.

- 단점
  - JPA의 장점을 살려 잘 사용하기 위해서는 학습 비용이 높고, 복잡한 쿼리를 사용할 때 불리하다.
  - 잘못 사용할 경우 SQL을 직접 사용하는 것보다 성능이 떨어질 수 있다.

JPA 사용하기

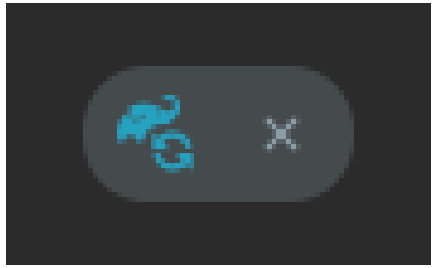
MyBatis 단원에서 만든 프로젝트를 JPA 방식으로 변경하기!

JPA 시작하기

- build.gradle 파일의 dependencies 항목 추가

```
runtimeOnly 'com.mysql:mysql-connector-j'
implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
```

- JPA를 사용하기 위해 “Load Gradle Changes” 버튼 클릭
  - Gradle 프로젝트 설정이 변경되었을 때 이를 IDE에 반영!



application.properties 수정

```
spring.application.name=spring-boot-jpa

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ [DataBase이름]?useUnicode=yes&characterEncoding=UTF-8&allowMultiQueries=true&serverTimezone=Asia/Seoul
spring.datasource.username=root
spring.datasource.password=1234

spring.jpa.database=mysql
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect


spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

* 데이터베이스명은 개발 환경마다 달라짐!

Entity 란?

- 데이터베이스 테이블과 매핑되는 Java 클래스

- 데이터베이스에 쓰일 필드와 여러 Entity 간의 관계를 설정하는 것
- @Entity 를 이용해 해당 클래스가 Entity임을 알려주고, JPA에 정의된 필드를 바탕으로 데이터베이스에 테이블을 만들 수 있다.

- 도메인(domain) 모델 이라고도 함

domain/UserEntity

```java
@Entity
@Table(name = "users")
@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String username;

    @Column(nullable = false, length = 100)
    private String email;

    @Column(name = "created_at", nullable = false, updatable = false, columnDefinition = "TIMESTAMP DEFAULT CURRENT_TIMESTAMP")
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
    }
}
```

Entity 관련 Annotation

- **@Entity**: 해당 클래스를 엔티티로 인식하도록 하며, 데이터베이스 테이블과 매핑됨
- **@Table**: 엔티티와 매핑할 데이터베이스 테이블 지정
- **@Id**: primary key 지정
- **@GeneratedValue**: primary key의 생성 전략
  - strategy=GenerationType.IDENTITY: 고유한 번호를 자동 생성
  - strategy=GenerationType.AUTO: 데이터베이스의 설정에 다라 적절한 전략을 자동으로 선택
- **@Column**: 엔티티 필드와 매핑할 테이블의 컬럼 지정
- **@PrePersist**: 엔티티가 데이터베이스에 저장되기 전에 필요한 초기화 작업 수행
  - ex. createdAt 필드 값을 현재 시각으로 설정

dto/UserDTO

```
import lombok.*;

@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UserDTO {
    private Long id;
    private String username;
    private String email;
  private int no;
}
```

Repository 란?

- Entity에 의해 생성된 테이블 데이터에 접근(CRUD)할 수 있도록 하는 인터페이스
- JpaRepository 를 상속받으면 테이블에 접근할 수 있는 기본적인 메소드 제공
  - findAll()
  - findById()
  - findBy컬럼명()
  - save()

repository/UserRepository

```
import codingon.spring_boot_jpa.domain.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

}
```

User Entity의 PK가 Long 타입

참고. Optional 이란?

- null 일 수도 있는 객체를 감싸는 Wrapper 클래스

- **Optional<T> option**
  - Option 변수 내부에는 null이 아닌 T 객체가 있을 수도 있고 null이 있을 수도 있다.
  - 즉, Optional 클래스는 여러 가지 API를 제공해 null일 수도 아닐 수도 있는 객체를 다룰 수 있다.

service/MainService

```java
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;

    public List<UserDTO> getAllUsers() {
        List<User> users = userRepository.findAll();
        List<UserDTO> userDTOs = new ArrayList<>();

        for (User user : users) {
            UserDTO userDTO = convertToDto(user);
            userDTOs.add(userDTO);
        }

        return userDTOs;
    }

    // User to UserDTO
    private UserDTO convertToDto(User user) {
        UserDTO dto = new UserDTO();
        dto.setId(user.getId());
        dto.setUsername(user.getUsername());
        dto.setEmail(user.getEmail());
        dto.setNo((int) (user.getId() + 100));

        return dto;
    }
}
```

controller/MainController

```
import codingon.spring_boot_jpa.dto.UserDTO;
import codingon.spring_boot_jpa.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping
    public List<UserDTO> listUsers() {
        return userService.getAllUsers();
    }

}
```

업그레이드. 사용자 조회

- 지금 수업에서 다룬 내용에 추가로 아래 작업들 진행해보자!
  - 사용자 이름으로 조회 (n명)
  - 검색어를 보냈을 때 사용자 이름/이메일에 특정 문자열이 포함된 모든 사용자 조회 (n명)
  - 이름이 존재하는지 조회

실습. 게시판 시스템

- MyBatis 로 만들었던 게시판 실습 JPA로 구현하기